

软件工程基础第 6 章

概要设计(Preliminary Design)

详细学习笔记

中南大学计算机学院任胜兵

2026 年 5 月 28 日

基于课件 PDF 全文（206 页）整理，包含所有核心知识点、图表说明、中英双语术语对照

目录

1 软件设计概述与质量模型	3
1.1 软件设计的基本概念	3
1.2 软件设计失败的案例	3
1.3 软件质量模型(Quality Model)	3
1.3.1 McCall 质量模型 (1977)	3
1.3.2 Boehm 的 Q-Model (1979)	3
1.3.3 ISO/IEC 9126 软件质量模型	4
1.4 软件设计与质量的关系	4
2 软件设计基本原理(Fundamental Design Principles)	5
2.1 抽象与求精(Abstraction and Refinement)	5
2.1.1 抽象(Abstraction)	5
2.1.2 求精(Refinement)	5
2.2 模块化和信息隐藏(Modularity and Information Hiding)	5
2.2.1 模块化(Modularity)	5
2.2.2 信息隐藏(Information Hiding)	6
2.3 模块独立性(Module Independence)	6
3 内聚度(Cohesion)——七种类型详析	6
4 耦合度(Coupling)——七种类型详析	7
5 面向数据流图的结构化设计(Structured Design)	9
5.1 结构化分析模型转换为结构化设计模型	9
5.2 软件结构图(Software Structure Chart, SC 图)	9
5.2.1 模块的六种类型	9
5.2.2 模块间的调用关系	10
5.2.3 数据或控制信息的表示	10
5.3 数据流变换分析法(Transform Analysis)	10
5.3.1 变换型数据流图的组成	10
5.3.2 传入/传出路径和变换中心的标识	10
5.3.3 第一级分解(First-level Factoring)	11
5.3.4 第二级分解(Second-level Factoring)	11
5.4 事务分析法(Transaction Analysis)	11
5.4.1 事务型数据流图的组成	11
5.4.2 事务中心的标识	12
5.4.3 第一级分解	12
5.4.4 第二级分解	12
5.5 软件结构图的改进(Improving Software Structure)	12
5.5.1 模块大小适中	12

5.5.2	模块扇入扇出合理	12
5.5.3	模块的作用域应在控制域内	13
6	面向 IDEF0 图的软件结构设计与 HIPO 图	13
6.1	面向 IDEF0 图的设计方法	13
6.2	HIPO 图(Hierarchy plus Input-Process-Output)	13
6.2.1	H 图(层次图(Hierarchy Chart))	13
6.2.2	改进的 IPO 图	13
6.3	基于 IDEF0 图的设计步骤	14
7	类设计原则(Class Design Principles)	14
7.1	单一职责原则(Single Responsibility Principle, SRP)	14
7.2	开闭原则(Open-Closed Principle, OCP)	14
7.3	里氏替换原则(Liskov Substitution Principle, LSP)	15
7.4	依赖倒置原则(Dependency Inversion Principle, DIP)	15
7.5	接口隔离原则(Interface Segregation Principle, ISP)	15
7.6	组装复用原则(Composition/Aggregation Reuse Principle, CARP)	15
8	面向对象设计(Object-Oriented Design, OOD)	16
8.0.1	Coad/Yourdon 方法的 OOD 模型	16
8.1	问题域部分设计(PDC Design)	16
8.2	人机交互部分设计(HIC Design)	17
8.2.1	设计活动	17
8.2.2	用户分类	17
8.2.3	用户及其任务脚本描述	17
8.2.4	命令层设计	17
8.2.5	详细交互设计准则	18
8.2.6	人机交互部分的类的设计	18
8.3	任务管理部分设计(TMC Design)	18
8.3.1	确定事件驱动型任务	18
8.3.2	确定时钟驱动型任务	18
8.3.3	其它活动	19
8.3.4	电梯控制系统的任务管理部分设计	19
8.4	数据管理部分设计(DMC Design)	19
8.4.1	主要活动	19
8.5	面向对象设计启发式规则	19
9	可复用面向对象设计模式(Reusable OO Design Patterns)	20
9.1	设计模式的描述与分类	20
9.1.1	GoF (Gang of Four) 23 种设计模式分类	20
9.2	创建型模式(Creational Patterns)	22

9.2.1	工厂方法模式(Factory Method Pattern)	22
9.2.2	抽象工厂模式(Abstract Factory Pattern)	22
9.3	结构型模式(Structural Patterns)	23
9.3.1	适配器模式(Adapter Pattern)	23
9.3.2	组合模式(Composite Pattern)	24
9.4	行为型模式(Behavioral Patterns)	24
9.4.1	迭代器模式(Iterator Pattern)	24
9.4.2	观察者模式(Observer Pattern)	25
10	应用框架(Application Frameworks)	25
10.1	应用框架的概念	25
10.2	应用框架的特点	25
10.3	应用框架的分类	26
10.4	应用框架的学习与使用	26
10.5	框架的开发方法	26
10.6	MVC 框架(Model-View-Controller)	26
10.7	库/设计模式/框架/体系结构复用比较	27
11	软件体系结构设计(Software Architecture Design)	27
11.1	体系结构设计与软件质量的关系	27
11.2	体系结构设计视图(Architecture Design Views)	28
11.2.1	4+1 视图模型	28
11.2.2	C4 模型视图	28
11.3	体系结构设计五原则	29
11.4	体系结构设计六大任务	29
11.5	体系结构模式风格(Architectural Pattern Styles)	29
11.5.1	MVC 模式(Model-View-Controller)	29
11.5.2	层次模式(Layered Architecture)	30
11.5.3	中心存储模式(Repository Architecture)	31
11.5.4	客户/服务器模式(Client-Server Architecture)	31
11.5.5	管道过滤器模式(Pipe and Filter Architecture)	32
11.5.6	微内核模式(Microkernel Architecture)	32
11.5.7	微服务模式(Microservices Architecture)	33
12	软件体系结构示例：铁路客运专线信息系统	34
12.1	案例概述	34
12.2	网络平台设计	34
13	本章核心概念总结	35
13.1	关键术语对照表	35
13.2	学习要点提示	35

1 软件设计概述与质量模型

1.1 软件设计的基本概念

[p.4]

软件设计的主要目的是为了得到便于实现的高质量软件。软件开发过程通常分为以下阶段：

需求分析 → 概要设计 → 详细设计 → 编码 → 程序代码

概要设计的基本目的是回答“概括地说，软件系统应如何实现”这一问题。

[p.5]

- 需求分析：软件系统必须“做什么”(What to do)
 - 软件设计：“如何做”(How to do)才能满足需求规格说明中规定的各项需求
- 从工程管理的角度，软件设计通常分为两步：
1. 概要设计(Preliminary Design)，又称初步设计或总体设计：关键是确定软件的总体结构，即确定软件系统的组成成分（子系统或模块）以及各组成成分之间的相互关系。
 2. 详细设计(Detailed Design)：对概要设计结果的进一步细化，确定软件系统各组成成分内部的数据结构和算法过程。

1.2 软件设计失败的案例

[p.7-8]

- 1991 年海湾战争爱国者导弹系统：软件设计问题导致跟踪计算不准确，系统运行时间越长，问题就越严重。
- 阿里亚娜 5 型火箭(Ariane 5)：系统软件试图将一个 64 位的数字塞入 16 位的空间，由此产生的溢出导致主计算机和备份计算机（它们都运行完全相同的软件系统）崩溃。
- 波音 787 重启问题：美国联邦航空管理局(FAA)下令，波音 787 飞机的动力系统必须要在运转 248 天后重启一次，以此来预防整架飞机丢失动力的情况发生。这种情况是由于发电机控制单元(GCUs)内部的一个软件计数器在连续供电 248 天后会溢出。

1.3 软件质量模型(Quality Model)

[p.9-12]

1.3.1 McCall 质量模型 (1977)

[p.10] 由 Jim McCall 于 1977 年提出，是第一个软件质量模型。

- 优点：明确了软件质量不只有一个维度，有多个维度。
- 缺点：不好使用，例如无法量化回答“质量有多好”。

1.3.2 Boehm 的 Q-Model (1979)

[p.11] 由 Boehm 于 1979 年扩展了 McCall 模型，由预定义的属性集和度量构成。

1.3.3 ISO/IEC 9126 软件质量模型

[p.12] 这是国际标准化组织的软件质量模型标准，包含 6 大质量特性和对应的子特性：

表 1: ISO/IEC 9126 软件质量模型

质量特性	含义	子特性
功能性(Functionality)	与一组功能及其指定的性质有关的一组属性	适合性、准确性、互操作性、依从性、安全性
可靠性(Reliability)	与在规定的一段时间和条件下，软件维持其性能水平的能力有关的一组属性	成熟性、容错性、易恢复性
可用性(Usability)	与一组规定或潜在用户为使用软件所需作的努力和对这样使用所作的评价有关的一组属性	易理解性、易学性、易操作性
效率(Efficiency)	与在规定的条件下，软件性能水平与所使用资源量之间的关系有关的一组属性	时间特性、资源特性
可维护性(Maintainability)	与进行指定的修改所需的努力有关的一组属性	易分析性、易改变性、稳定性、易测试性
可移植性(Portability)	与软件可从某一环境转移到另一环境的能力有关的一组属性	适应性、易安装性、一致性、易替换性

1.4 软件设计与质量的关系

[p.13-14]

软件产品质量公式：

$$\text{软件产品质量} \approx (\text{需求分析质量} \times \text{设计质量} \times \text{编码质量}) \times \text{验证质量}$$

设计阶段的主要质量问题：

1. 需求分析不充分，导致设计成为无本之木
2. 设计不重视质量，直接编码，导致设计不充分
3. 设计过度重视某些质量属性，导致设计太复杂，顾此失彼
4. 设计漂移，导致设计质量下降

软件设计可以提高软件质量:

- 质量往往具有全局性，没有设计看不到全局
- 软件设计原则有助于提高软件质量
- 软件设计模型有助于质量分析

软件设计可以提高软件开发效率:

- 软件设计可以避免一次考虑太多东西
- 软件设计可将复杂的软件转换成简单模块

2 软件设计基本原理(Fundamental Design Principles)

[p.17]

2.1 抽象与求精(Abstraction and Refinement)

[p.18-19]

2.1.1 抽象(Abstraction)

抽象是一种常用的思考和解决问题的方式，即抽取事物的本质的共同特性而暂时避开不必要的低层细节。

三种抽象方式:

1. **过程抽象(Procedural Abstraction)**: 具有特定功能的一个命名的指令序列。(如: 二维图形创建)
2. **数据抽象(Data Abstraction)**: 描述数据对象的一个命名的数据集合。(如: “图画”数据对象)
3. **控制抽象(Control Abstraction)**: 包含了一种程序控制机制而无须刻画其内部细节。(如: 操作系统中的“同步信号量”)

2.1.2 求精(Refinement)

由 N. Wirth 最初提出的一种自顶向下设计策略，其主要思想是: 将某个宏观功能不断分解，逐步确定过程细节，直至用程序设计语言描述的算法实现为止。

- 抽象使得设计人员能够避开过早地陷入细节之中刻画过程和数据
- 求精能够帮助设计人员随着设计过程的深入而不断呈现更低层次的信息

2.2 模块化和信息隐藏(Modularity and Information Hiding)

[p.20-21]

2.2.1 模块化(Modularity)

软件应该分解成可单独命名的且可访问的部件，这些部件称为**模块(Module)**。

分而治之(Divide and Conquer): 把一个大问题分解成若干小问题来解决时将会更加容易。

模块数量与开发代价的关系呈 U 型曲线:

- 模块太少：每个模块过于复杂，开发代价高
- 模块太多：接口开发代价高
- 最小代价区 M：模块数目适中，总开发代价最低

2.2.2 信息隐藏(Information Hiding)

由 Parnas 倡导的“信息隐藏”是指模块中所包含的信息（包括数据和过程）对不需要这些信息的其它模块是不可访问的。

- 抽象有助于定义组成软件的过程（或信息）实体
- 隐藏定义并加强了对模块内部访问的约束，有助于分离模块的实现者和使用者

2.3 模块独立性(Module Independence)

[p.22]

模块独立性是模块化、抽象和信息隐藏的直接产物，其基本含义是每一个模块只完成功能需求中的一个特定的子功能，而且从程序结构的其它部分来看这一模块只具有一个简单的接口。

- 模块的功能独立性可以使得模块既容易开发又容易维护
- 模块独立性有两个定性的度量标准：内聚度(Cohesion)和耦合度(Coupling)

3 内聚度(Cohesion)——七种类型详析

[p.23-30]

内聚度(Cohesion)是指模块内部各成分联系紧密的程度。通常，内聚度越高，模块的独立性就越强。

G. Myers 定义了七种类型的内聚，按照内聚程度从高到低的顺序排列如下：

表 2: 内聚度七种类型对比

等级	类型	定义	示例/特点
高（最优）	功能内聚(Functional)	模块只完成一个单一的功能或只实现一个单一的目标	这样的模块很容易被复用，也很容易维护 [p.30]
高	信息内聚(Informational)	模块完成多个功能，每个功能有各自入口点和独立代码，但都依赖于同样的数据结构（数据相关）	前一成分的输出是后一成分的输入。如：输入系数、高斯消元求根、输出根 [p.29]
中	通信内聚(Communicational)	模块完成多个需按一定步骤依次完成的功能，且这些功能都依赖于相同的公用数据（数据相关）	如：一个模块的各功能部分都使用相同输入数据和（或）产生相同输出数据。用数据流图划分模块通常得到通信内聚 [p.28]
中	过程内聚(Procedural)	模块完成多个需要按一定的步骤依次完成的功能（过程相关）	如：用程序流程图划分模块时，将一部分划出组成模块便形成过程内聚。比时间内聚内聚度高 [p.27]
低	时间内聚(Temporal)	模块完成多个具有时间相关性的功能	如：初始化模块、故障处理模块（保留现场、报警）。内部功能通常弱相关，但与其他模块功能强相关。各部分可以任意次序执行 [p.26]
低	逻辑内聚(Logical)	模块完成多个逻辑上相近或相关的功能，每次调用只选择其中一个功能执行（多个逻辑功能交织）	如：完成所有类型输入输出的模块。需要传递控制信息（控制耦合） [p.25]
最差	偶然内聚(Coincidental)	模块内部各成分在功能上不相关或关系松散	如：将多段在不同子程序中重复出现的语句组成一个模块。不易理解、不易维护、不易复用 [p.24]

设计原则：应该尽可能避免使用偶然内聚等低级内聚的模块，争取高级内聚的模块，以提高模块的独立性。

4 耦合度(Coupling)——七种类型详析

[p.31-38]

耦合(Coupling)是模块之间相互关联紧密的程度。一般地，模块的耦合度越低，模块的独立

性越强。

模块之间的耦合程度从低到高也分为七种：

表 3: 耦合度七种类型对比

等级	类型	定义	特点与问题
最优（最低）	非直接耦合(No Direct Coupling)	两个模块都不需要另一模块的存在而能独立工作	耦合度最低，但在一个软件系统中不可能所有模块间都没有任何联系 [p.38]
优	数据耦合(Data Coupling)	所有参数都是简单变量，或者虽有结构变量但被调用模块需要该结构中的所有数据	如果系统中只有数据耦合，则其它耦合存在的所有问题都不会存在，维护就会很容易 [p.37]
中	标记耦合(Stamp Coupling)	传递的参数是结构变量（如记录名、数组名、文件名等），但被调用模块只操纵参数中的一部分数据	无法控制对数据的存取，易导致对数据的非法存取 [p.36]
中偏高	控制耦合(Control Coupling)	一个模块传递控制信息给另一个模块，显式地控制其逻辑	被调用模块必须知道调用模块的内部结构和逻辑，可复用性降低。通常与逻辑内聚有关 [p.35]
高	外部耦合(External Coupling)	一组模块访问同一单一类型全局变量或全局数组	如 C 语言中访问 extern 类型外部变量。类似于公共耦合但不存在依赖于数据结构内部各项的物理安排 [p.34]
高	公共耦合(Common Coupling)	两个模块都能存取同一公共数据环境（全局数据结构、共享通信区、内存公共覆盖区、文件等）	松散公共耦合：一个只送一个只取；紧密公共耦合：两个都既取又送。难以理解、带来副作用、难以维护、难以复用、易导致安全问题 [p.33]
最差	内容耦合(Content Coupling)	一个模块直接引用另一个模块的内容（数据、语句、标号等）	一般出现在汇编语言程序中。一个修改则另一个也必须修改，不易复用 [p.32]

设计原则：应该尽量使用**数据耦合**，必要时使用**标记耦合**，少用**控制耦合**，限制使用**公共耦合**，最好不要使用**内容耦合**。

5 面向数据流图的结构化设计(Structured Design)

[p.39-60]

面向数据流图的设计方法是一种结构化设计方法(Structured Design Method)。其基本步骤为：

1. 研究、分析和审查数据流图，确保数据流图符合实际，必要时进一步精化数据流图
2. 确定数据流图的类型：变换型数据流(Transform Flow)或事务型数据流(Transaction Flow)
3. 依据数据流图类型采用变换分析法或事务分析法导出系统初始的软件结构
4. 依据软件设计原理和优化策略改进系统初始的软件结构，形成最终的软件结构

5.1 结构化分析模型转换为结构化设计模型

[p.40]

表 4: 分析模型与设计模型的转换

分析模型成分	设计模型成分
数据流图(DFD)、ER 图、数据词典	软件结构设计（概要设计）
状态转换图(STD)、控制规格说明	接口设计
处理规格说明	过程设计（详细设计）
数据词典	数据结构设计

5.2 软件结构图(Software Structure Chart, SC 图)

[p.41-44]

在结构化设计方法中，软件结构图是软件系统模块层次结构，反映了整个系统功能及其之间的关系。

软件结构图的主要内容有：

1. 模块(Module)
2. 模块间的调用关系(Calling Relationship)
3. 模块之间传递的信息(Information Passing)

5.2.1 模块的六种类型

[p.42]

在软件系统的软件结构图中，有 6 种类型的模块：

1. 传入模块(Input Module)：从下层模块接收输入数据
2. 传出模块(Output Module)：向下层模块发送输出数据
3. 变换模块(Transform Module)：将输入数据转换为输出数据
4. 协调模块(Coordinating Module)：协调和控制其他模块
5. 源模块(Source Module)：数据源
6. 漏模块(Sink Module)：数据终点

在软件结构图中，模块用方框来表示，并用名字标识该方框。

5.2.2 模块间的调用关系

[p.43]

模块间的调用关系主要有三种：

1. 顺序调用(Sequential Call)：A 顺序调用 B、C 和 D
2. 选择调用(Selective Call)：A 调用 B 或 C 和 D（用菱形表示条件）
3. 循环调用(Iterative Call)：A 循环调用 B、C 和 D（用弧形箭头表示循环）

方框之间的箭头表示模块之间的调用与被调用关系。模块间调用的次序，习惯上是从左至右。

5.2.3 数据或控制信息的表示

[p.44]

在软件结构中，模块传递的信息用带名称的短线箭头来表示。箭头方向代表信息传递的方向。

- 箭头线尾带空心圆圈：表示传递的是数据(Data)
- 箭头线尾带实心圆圈：表示传递的是控制信息(Control)

5.3 数据流变换分析法(Transform Analysis)

[p.45-50]

数据流变换分析法是一种将变换型数据流图映射为变换型软件结构图的软件系统设计方法。

5.3.1 变换型数据流图的组成

[p.45]

变换型数据流图一般呈线性形状，大致由三部分组成：

1. 传入路径(Input Path/Incoming Flow)
2. 变换中心(Transform Center)
3. 传出路径(Output Path/Outgoing Flow)

变换型软件结构图大致由四部分模块组成：

1. 输入模块(Input Module)
2. 中心加工模块(Central Processing Module)
3. 输出模块(Output Module)
4. 主控模块(Main Control Module)

5.3.2 传入/传出路径和变换中心的标识

[p.46]

采用试探的方法来确定系统的变换中心和传入/传出路径：

- 从数据流图的物理输入始端出发，沿数据流一步步往输出末端移动，直到遇到的数据流不能再被看作是系统的输入数据为止，则其前一个数据流便是系统的**逻辑输入**
- 从数据流图的物理输出末端一步步往输入始端移动，直到遇到的数据流不能再被看作是系统的输出数据为止，则其后一个数据流便是系统的**逻辑输出**
- 从物理输入始端至系统的逻辑输入构成系统的**传入路径部分**
- 从物理输出末端至系统的逻辑输出构成系统的**传出路径部分**
- 逻辑输入和逻辑输出之间的部分构成**变换中心部分**

5.3.3 第一级分解(First-level Factoring)

[p.48]

将数据流图映射为一个能为信息的输入、变换和输出提供控制的特定结构：

- 数据输入控制模块
- 数据计算控制模块（变换中心）
- 数据输出控制模块

5.3.4 第二级分解(Second-level Factoring)

[p.49]

将数据流图中的每一个加工处理映射为软件结构图中一个模块：

- 对于**传入路径**中的加工处理：从变换中心的边界开始，沿着传入路径向外移动，把传入路径中遇到的每个加工处理映射成软件结构图中输入信息处理控制模块直接或间接调用的模块
- 对于**传出路径**中的加工处理：从变换中心的边界开始，沿着传出路径向外移动，把传出路径中遇到的每个加工处理映射成软件结构图中输出信息处理控制模块直接或间接调用的模块
- 对于**变换中心**的加工处理：直接映射为变换流控制模块直接调用的模块

5.4 事务分析法(Transaction Analysis)

[p.51-56]

事务分析法是将**事务型数据流图**映射为**事务型软件结构图**的软件系统设计方法。

5.4.1 事务型数据流图的组成

[p.51]

事务型数据流图一般呈辐射状，由三部分组成：

1. 至少一条**接收路径**(Reception Path)
2. 一个**事务中心**(Transaction Center)
3. 若干条**动作路径**(Action Paths)

事务型软件结构图大致由四部分模块构成：

1. **事务接收模块**(Transaction Reception Module)
2. **事务调度模块**(Transaction Dispatch Module)
3. **事务处理控制模块**(Transaction Handling Control Module)（可能有多个）
4. **总控模块**(Main Control Module)

5.4.2 事务中心的标识

[p.52]

事务中心一般位于几条动作路径的起始点上。动作路径具有自己的结构特征，可能是事务型的，也可能是变换型的。在标识事务中心的同时，还应确定每条动作路径的结构特征。

5.4.3 第一级分解

[p.54]

将事务型的数据流图映射到包含一个事务接收分支和一个事务分类处理分支的软件结构上。

5.4.4 第二级分解

[p.55]

- 从事务中心开始，沿接收路径的变换被映射成事务接收模块直接或间接调用的模块
- 数据流图的每一条动作路径映射成与其自身的数据流图类型相一致的软件结构，其方法是变换分析法或事务分析法

5.5 软件结构图的改进(Improving Software Structure)

[p.57-60]

在改进软件结构时应该记住“一个不能工作的最佳设计”是毫无价值的。设计人员应该致力于设计能够满足所有功能和性能要求，而且按照设计原理和改进规则衡量是值得接收的软件。

5.5.1 模块大小适中

[p.58]

- 模块大小影响模块的可理解性
- 限制模块的大小也是减少软件复杂性的手段之一
- 对于篇幅较大的模块，检查其包含的功能，从中分离出一些功能构成软件结构图中的同层或下一层的其它模块
- 对于篇幅较小的模块，应考虑是否可以同它的上层调用模块合并

5.5.2 模块扇入扇出合理

[p.59]

- **扇出(Fan-out)**：模块直接调用多少其它模块。扇出过大表明该模块过分复杂，需要协调和控制过多的下层模块
- **扇入(Fan-in)**：共有多少模块直接调用本模块。扇入过大（且不是公用模块）一般说明该模块可能具有多个功能
- 经验表明，良好的软件结构图，上层模块（主要是控制模块）往往具有较高的扇出，底层的模块（主要是功能型模块）具有较高的扇入，呈**两头小、中间大的清真寺状**

5.5.3 模块的作用域应在控制域内

[p.60]

- **作用域(Scope of Effect)**: 模块中判定的作用范围, 即所有受这个判定影响的模块
- **控制域(Scope of Control)**: 模块本身及其直接或间接调用的模块
- 如果模块的作用域不在控制域之内, 则会增加模块间数据的传递量, 使模块之间出现控制耦合

6 面向 IDEF0 图的软件结构设计与 HIPO 图

[p.61-65]

6.1 面向 IDEF0 图的设计方法

[p.61]

IDEF0 是一种结构化设计方法, 它以系统的功能模型和信息结构(数据)为基础设计系统的软件结构。

由于 IDEF0 图按照自顶向下逐层对系统进行分解, 并且全面描述了系统每一功能的输入、输出、约束和机制。因此在概要设计时, 一般可按照 IDEF0 图的分解层次, 逐层将其转换成软件结构。

6.2 HIPO 图(Hierarchy plus Input-Process-Output)

[p.62-63]

HIPO 图是美国 IBM 公司 70 年代发展起来的表示软件结构的工具。HIPO 图由 **H 图**和 **IPO 图**构成。

6.2.1 H 图(层次图(Hierarchy Chart))

[p.62]

用来描述软件系统模块的层次结构:

- 用矩形框代表一个模块
- 用矩形框之间的连线表示模块之间的调用关系

例如: 仓库管理系统由进仓处理、出仓处理、数据处理等模块组成, 其中数据处理模块又细分为查询、统计、报表打印等子模块。

6.2.2 改进的 IPO 图

[p.63]

IPO 图主要用来对 H 图中的每个模块进行详细描述。根据设计的需要, 对 IPO 图进行了改进, 用来描述:

- 模块输入/输出数据
- 处理功能

- 模块间的调用情况（被调用模块、调用模块）
- 局部数据元素
- 注释

IPO 图包含的字段：系统名称、模块名称、模块编号、被调用关系、调用关系、输入、输出、处理描述、局部数据元素、注释、作者、日期。

6.3 基于 IDEF0 图的设计步骤

[p.64]

1. 找出该层 IDEF0 图的父图，搞清父、子图之间的输入/输出关系和控制关系
2. 以父图中的活动为上层模块，活动分解得到的子图中的活动作为直接调用的下层模块，画出系统的单层结构图
3. 根据 IDEF0 图各个活动的输入/输出数据、控制信息等，确定模块的接口
4. 综合所有层次的结构图，便可得到系统初始的层次图
5. 通过反复理解全部 IDEF0 图的内容和含义，对系统初始的层次图进行必要的调整、修改、分解或合并，形成最终的层次图

7 类设计原则(Class Design Principles)

[p.66-76]

在面向对象软件的设计过程中，人们总结了许多旨在提高类设计质量的原则。依据实际问题的特点，灵活运用这些设计原则，可以有效地提高面向对象软件的设计质量。

7.1 单一职责原则(Single Responsibility Principle, SRP)

[p.67-68]

所谓单一职责原则，要求的是一个类应该仅有一个引起它变化的原因。

- 当需求变化时，类的职责就会因此而变化，即类的职责是引起类变化的原因
- 如果一个类承担的职责过多，会导致设计脆弱，即当设计因一个职责的变化而发生改变时会遭到意想不到的破坏，影响这个类完成其它职责的能力
- 分离职责可能会导致不必要的设计复杂性，设计人员应在设计过程中权衡利弊，审慎地使用单一职责原则

示例：Rectangle 类同时承担了计算面积（Computational Geometry）和绘制图形（Graphical Application）两种职责，违反了 SRP。应分离为 Geometric Rectangle（负责面积计算）和 Graphical Rectangle（负责绘图）两个类。

7.2 开闭原则(Open-Closed Principle, OCP)

[p.69-70]

开闭原则讲的是：一个软件设计应当对扩展是开放的(Open for extension)，但对于修改是封闭的(Closed for modification)。

背景故事：在《西游记》中，孙悟空大闹天宫时要求“皇帝轮流做，明年到我家”。太白金星建议“与他籍名在录……一则不动众劳师，二则收仙有道也。”玉皇大帝采纳该建议，授孙悟空“弼马温”官职。这是在不修改原有天宫体系（对修改封闭）的情况下，通过增加新官职（对扩展开放）来解决问题的例子。

实现方式：引入抽象类(Abstract Class)或接口(Interface)，使 Client 依赖于抽象而不是具体实现。

7.3 里氏替换原则(Liskov Substitution Principle, LSP)

[p.71-72]

若对每个类型为 S 的对象 O_1 ，都有类型为 T 的对象 O_2 ，使得在所有针对 T 编写的程序 P 中，用 O_1 替换 O_2 后，程序 P 的行为没有变化，则 S 是 T 的子类型。

里氏替换原则要求子类型必须能够替换掉其基类型。这是采用继承来完成复用的基础。只有在派生类替换其基类后，软件的功能不受影响时，基类才真正被复用。

反例：Square 继承 Rectangle。Rectangle 有 setWidth() 和 setHeight() 方法，但 Square 的宽高必须同时变化，导致 Square 不能真正替换 Rectangle。

解决方案：提取公共部分，使用更抽象的基类。

7.4 依赖倒置原则(Dependency Inversion Principle, DIP)

[p.73]

依赖倒置原则要求：

- 高层模块不应依赖于低层模块，两者都应该依赖于抽象
- 抽象不应依赖于细节，细节应该依赖于抽象

示例：Button 类不应直接调用 Lamp 类的 turnOn()/turnOff() 方法，而应依赖于抽象的 ButtonServer 接口。这样 Button 可以控制任何实现了 ButtonServer 接口的设备（灯、电视、空调等）。

7.5 接口隔离原则(Interface Segregation Principle, ISP)

[p.74-75]

接口隔离原则讲的是：使用多个专门的接口比使用单一的总接口要优越。

- 强迫客户程序依赖于它们不用的方法会导致客户程序面临因这些未使用方法的改变所带来的变更
- 由于单一的总接口由不同的客户程序使用，导致不相关的客户程序的耦合，使得一个客户程序要求对接口的改变影响到其它所有客户程序

示例：TimedDoor 需要实现 TimerClient 接口的 timeout() 方法，但不希望 Door 类本身依赖 TimerClient。使用 DoorTimerAdapter 适配器来实现接口隔离。

7.6 组装复用原则(Composition/Aggregation Reuse Principle, CARP)

[p.76]

在面向对象的设计中，有两种实现复用已有设计和实现的方法：

1. **继承(Inheritance)**: 白盒复用, 编译时确定

2. **组装(Composition/Aggregation)**: 黑盒复用, 运行时确定

组装复用原则指在新的对象里尽量使用已有的类对应的对象并通过委派达到复用已有功能的目的。优先使用对象组合(组装)而非类继承。

示例: 客货两用车通过组装”客车”和”货车”来实现, 而非通过多重继承。

8 面向对象设计(Object-Oriented Design, OOD)

[p.77-98]

面向对象设计是在对象建模的基础上进行逐渐扩充的过程。

- 在对象建模中, 是以**问题域**为中心
- 面向对象设计要在**软件系统环境**(即解空间)中解决对象建模中要完成的事情
- OOA 以问题域为中心确定”做什么”(What to do)
- OOD 以软件系统实现环境为中心确定”如何做”(How to do)

8.0.1 Coad/Yourdon 方法的 OOD 模型

[p.77]

OOA 模型的五个层次: 对象层、结构层、主题层、属性层、服务层。

OOD 模型的四个部分:

1. **问题域部分**(Problem Domain Component, PDC)
2. **人机交互部分**(Human-Computer Interaction Component, HIC)
3. **任务管理部分**(Task Management Component, TMC)
4. **数据管理部分**(Data Management Component, DMC)

8.1 问题域部分设计(PDC Design)

[p.78-80]

在 OOA 模型中没有考虑与实现有关的问题, 例如最终用户如何与待开发的系统交互, 对象的响应性能等。

在 OOA 模型中有许多方面可以直接不加修改地作为面向对象设计的问题域部分, 这正是面向对象设计的优越之处。

OOA 模型的修改:

[p.79]

1. 调整需求
2. 重用已有的类
3. 将问题域有关的类组成一组, 并建立一个公共的协议
4. 对继承进行调整, 以利于实现
5. 提高性能

电梯控制系统示例:

[p.80]

- 增加一个中央控制器(电梯控制器)来控制 and 协调所有电梯的动作(增加监控需求)
- 增加”电梯监视器”对象/类

- 每个类中增加一个”自检检测”操作用来向”电梯监视器”对象/类报告各自的运行状态信息
- 增加一个”电梯调度”对象/类来实现所有电梯的调度，替代传感器检测（提高性能）

8.2 人机交互部分设计(HIC Design)

[p.81-89]

人机交互部分突出人如何命令系统以及系统如何向用户提供交互信息。

设计优先级：首先是人，其次是任务，然后再是有关的工具。

人机交互部分在系统行为和用户界面之间架起了一座桥梁。

8.2.1 设计活动

[p.82]

1. 用户分类
2. 用户及其任务脚本描述
3. 命令层设计
4. 详细交互设计
5. 人机交互部分的类的设计

8.2.2 用户分类

[p.83]

- 按技能水平分：初学者、临时人员、中级人员和高级用户等
- 按组织分：行政人员、办公人员、职员等
- 按不同组的成员身份分：职员、顾客等

需要考虑的问题：如何工作？完成何种工作？

8.2.3 用户及其任务脚本描述

[p.84] 需要描述的内容包括：用户类型、用户使用系统欲达到的目的、用户的特征（年龄、性别、受教育程度和限制因素等）、关键的成功因素（需求、爱好、习惯等）、技能水平、完成本职工作的任务脚本等。

8.2.4 命令层设计

[p.85]

- 命令层是用过程抽象（相同过程聚类）所组织的可用服务的一种体现
- 研究现有的用户交互活动的含义和准则，确定初始的命令层次
- 细化命令层：排列命令（如按照用户工作步骤排列，或把使用最频繁的服务放在前边）、整体-部分组合、命令层宽度与深度对比（不要超过人类短期记忆的局限）、减少操作步骤（如提供操作捷径）

8.2.5 详细交互设计准则

[p.86]

- 一致性(Consistency): 使用一致的术语、一致的步骤、一致的动作
- 及时提供反馈信息: 应简短、直接、及时从用户角度作出提示
- 提供撤消命令(Undo)
- 减少用户记忆负担: 每步给出足够的提示
- 易学, 富有吸引力: 提供联机帮助, 充分利用图形用户界面的优越性

8.2.6 人机交互部分的类的设计

[p.87]

人机交互的形式多种多样: 对话、菜单、窗口、按钮、图符、表格、命令语言等。

设计目的是决定使用哪种方式的人机交互, 并标识有关的人机交互对象/类。

注意事项:

[p.88]

- 不是改变用户行为去适应系统的人机交互界面, 而是反之
- 一个好的用户界面总是能在工作中引导用户完成其工作, 而不是强迫用户按某种特定方式工作
- 最好由用户开发的场景或使用用例来驱动用户界面

8.3 任务管理部分设计(TMC Design)

[p.90-95]

所谓任务(Task), 是进程(Process)的别名, 指执行一系列活动的一段程序, 是计算机可调度的一些独立的实体。

任务管理部分的设计主要是标识一些对象/类, 负责处理并发、中断、调度(操作系统级)以及其它有关特定平台的一些问题。

8.3.1 确定事件驱动型任务

[p.91]

事件驱动型任务由事件触发(事件常是表明某些数据到达的信号), 这些任务可能负责与设备、屏幕窗口、子系统或其它任务等通信。

工作流程: 数据线或其它数据源中断 → 任务苏醒 → 阅读数据 → 处理数据 → 结束处理 → 任务睡眠。

8.3.2 确定时钟驱动型任务

[p.92]

时钟驱动型任务按特定的时间间隔被触发进行某些处理。这类任务适用于某些人机接口、子系统、任务或其它系统周期性地通信。

工作流程: 任务设置睡眠时间后转入睡眠 → 系统中断 → 通知任务 → 任务苏醒 → 任务处理 → 结束处理。

8.3.3 其它活动

[p.93]

- 识别优先任务和关键任务
- 识别任务协调者
- 审查各个任务
- 定义各任务

模型中对象/类的每个操作最终总要被分配给某个任务。如果某个操作可以分解成多个任务，则应修改该操作的名称，以使每个操作映射到一个任务中。

8.3.4 电梯控制系统的任务管理部分设计

[p.94-95]

- **中断处理任务：**召唤按钮中断任务、目的地按钮中断任务、到达中断任务、电梯就绪中断任务
- **寄存器访问任务：**输入寄存器访问任务、输出寄存器访问任务
- 每个中断任务和访问任务之间的协同工作过程（详细交互流程）

8.4 数据管理部分设计(DMC Design)

[p.96-97]

数据管理部分的对象/类用来对系统生成的永久数据进行访问和管理。

设置数据管理部分可以将特定的数据管理技术从问题域部分分离开来，有利于系统的维护。

8.4.1 主要活动

[p.97]

1. **选择数据存储管理模式：**普通文件管理系统、关系数据库管理系统(RDBMS)和面向对象数据库管理系统(OODBMS)
2. **数据存放格式设计**（一项重要的工作便是消除数据冗余）
3. **设计数据管理操作和属性**

电梯控制系统：所有对象/类中的数据都驻留在内存中，在系统掉电后不必保留它们，因而没有数据管理部分。

8.5 面向对象设计启发式规则

[p.98]

- **简单性(Simplicity)：**保持对象/类的简单性、保持方法的简单性和保持消息协议的简单性
- **可理解性(Understandability)**
- **内聚性(Cohesion)：**一是单个方法的内聚性；二是封装在类中的数据和方法的内聚性；三是整个类层次结构的内聚性
- 将设计的易变性降到最低

9 可复用面向对象设计模式(Reusable OO Design Patterns)

[p.99-149]

面向对象设计模式最初出现于 70 年代末 80 年代初。1987 年 W. Cunningham 和 K. Beck 引入了建筑师 C. Alexander 的模式概念。

面向对象设计模式是普通面向对象设计问题的解决方案，这类问题以一组交互类的形式出现，用户根据需要定制这些交互类以形成专门的设计。

作用：设计模式不仅使人们可以更加方便地复用成功设计方案，也能提高已有系统的文档管理和系统维护的有效性。

9.1 设计模式的描述与分类

[p.100-103]

一个设计模式事实上是系统地命名、解释和评价某一重要的可重现的面向对象设计方案。所有设计均可从四个方面加以描述：

1. **模式名称(Pattern Name)：**用一至两个词描述设计问题、解决方案和效果。模式名称便于使用者在更高的抽象层次上进行设计并交流有关设计思想。
2. **问题描述(Problem Description)：**指明使得设计模式可以被应用所必须存在的环境条件。它解释了设计问题及其背景。
3. **解决方案(Solution)：**描述了设计方案的组成部分，它们之间的关系及各自的职责和协作方式。
4. **效果(Consequences)：**描述应用设计模式后的结果及使用模式应权衡的问题。

9.1.1 GoF (Gang of Four) 23 种设计模式分类

[p.101-103]

表 5: GoF 设计模式完整分类表

类别	模式名称	意图简要说明
创建型(Creational)	抽象工厂(Abstract Factory)	提供创建相关的或相互依赖的一族对象的接口而无需指定具体的类
	生成器(Builder)	将一个复杂对象的创建与它的表示分离, 使得同样的构建过程可以创建不同的表示
	工厂方法(Factory Method)	定义一个用于创建对象的接口, 由子类决定实例化哪一个类
	原型(Prototype)	使用一个原型指定要创建的类的类型, 通过拷贝这个原型得到新的对象
	单件(Singleton)	保证一类仅有一个实例, 并提供一个全局性的访问点
结构型(Structural)	适配器(Adapter)	将一个类的接口转换成用户希望得到的另一种接口, 使原来由于接口不相容而不能一起工作的类可一起工作
	桥(Bridge)	将类的抽象部分与它的实现部分分离, 使它们可以相互独立地变化
	组合(Composite)	将对象组成树结构来表示局部和整体的层次关系, 使单个对象和组合对象的使用具有一致性
	装饰(Decorator)	动态地给一个对象添加新功能
	外观(Façade)	给一个子系统的所有接口提供一个统一接口, 使子系统便于使用
	轻量(Flyweight) 代理(Proxy)	运用共享技术有效地支持大量细粒度对象 为其他对象提供一种代理以控制对这个对象的访问
行为型(Behavioral)	职责链(Chain of Responsibility)	将多个接收请求的对象连成一条链, 并沿链传递请求, 直到有一个对象处理它为止
	命令(Command)	将一个请求命令封装为一个对象, 便于将不同的请求参数化; 对请求排队或记录请求日志, 以及支持可撤消请求的操作
	解释器(Interpreter)	给定一种语言, 定义它的文法的一种表示, 并定义一个解释器, 该解释器使用该表示解释语言中的句子
	迭代器(Iterator)	提供一种顺序访问一个聚合对象中各元素又不暴露对象内部表示的方法
	中介者(Mediator)	定义一个中介对象封装一系列对象的交互
	备忘录(Memento)	在不破坏封装的条件下, 获得一个对象的内部状态并将它外部化, 从而在以后可使对象恢复到这个状态
	观察者(Observer)	定义对象一对多的依赖关系, 当一个对象的状态发生改变时, 所有依赖于它的对象均得到通知并被自动更新
	状态(State)	允许一个对象在其内部状态改变时改变它的行为。对象看起来似乎能修改它的类
	策略(Strategy)	定义一系列算法, 把它们均封装起来, 并且使它们可相互替换。它使算法可独立于使用者而变化
	模板方法(Template Method)	定义一个操作中的算法的骨架, 而将一些步骤延迟到子类中, 从而使子类可以不改变算法的结构即可重定义该算法的某些特定步骤
	访问者(Visitor)	描述一个作用于某对象结构中各元素的操作, 从而可以在不改变被操作元素类的条件下定义新的操作

9.2 创建型模式(Creational Patterns)

[p.104-117]

创建型模式帮助系统独立于对象的产生、组合和表示。

作用：一方面均将关于系统使用哪些具体的类的信息封装起来；另一方面隐蔽了这些具体类的实例是如何被创建和放在一起的。因此，创建型模式在” 什么” 被创建、” 怎样” 被创建、” 谁” 创建它以及” 何时” 创建等方面带来了很大的灵活性，有利于设计可复用的软件成分。

9.2.1 工厂方法模式(Factory Method Pattern)

[p.105-111]

结构：由四个角色组成：

- **Product**（产品接口）：定义工厂方法生成的对象的接口
- **ConcreteProduct**（具体产品）：实现 Product 接口
- **Creator**（创建者接口）：声明工厂方法，返回 Product 类型对象
- **ConcreteCreator**（具体创建者）：重定义工厂方法以返回适当的 ConcreteProduct 实例
- **Client**（客户）：仅使用由 Creator 和 Product 声明的接口

使用条件：

- 一个类不知道它所必须创建的对象类
- 一个类希望由其子类确定它创建的对象
- 类将创建对象的职责委派给多个帮助者子类中的一个，且希望将哪一个帮助者子类是代表者这一信息局部化

使用效果：

- 比直接产生对象更灵活。给子类一个挂钩(Hook)，以便在子类中扩展
- 连接平行类层次。当一个类将它的一些职责委托给一个独立的类的时候，就会产生平行类层次
- 工厂方法模式在遵循**依赖倒置原则**方面有重要意义，使得高层策略模块在创建类的实例时无需依赖具体的类
- 工厂方法模式会带来复杂性

9.2.2 抽象工厂模式(Abstract Factory Pattern)

[p.112-117]

结构：由以下角色组成：

- **AbstractFactory**（抽象工厂）：声明创建抽象产品对象的操作接口
- **ConcreteFactory**（具体工厂）：实现创建具体产品对象的操作
- **AbstractProduct**（抽象产品）：声明一种产品对象的接口
- **ConcreteProduct**（具体产品）：定义将被相应的具体工厂类创建的产品对象
- **Client**（客户）：仅使用由抽象工厂类和抽象产品类声明的接口

关键概念：

- **产品等级结构(Product Hierarchy)**：产品的继承结构（如 AbstractProductA 与其子类）
- **产品族(Product Family)**：由同一个工厂生产的位于不同产品等级结构中的一组产品

使用条件：

- 系统应与如何创建、组合和表示其产品无关
- 系统应由多个产品族之一来配置
- 强调一系列相关产品对象的设计以便联合使用
- 提供一个产品类库，但只想显示它们的接口

使用效果：

- 隔离了具体的类
- 使产品族间的转换容易进行
- 容易实现一个应用一次只使用同一个系统中的产品对象
- 难以扩展”抽象工厂”以支持新种类的产品（不符合开闭原则）

9.3 结构型模式(Structural Patterns)

[p.118-133]

结构型模式涉及如何组合类和对象构成更大的结构。一种方法是采用继承机制来组合接口或实现来形成更大的结构；另一种方法通过对象组合方式对一些对象进行组合来形成。由于对象组合可以在运行时刻改变，而继承机制为静态类组合，因而对象组合方式具有更大的灵活性。

9.3.1 适配器模式(Adapter Pattern)

[p.119-126]

结构：两种实现方式：

1. 利用继承机制（类适配器）：Adapter 继承 Adaptee 并实现 Target 接口
2. 利用组合机制（对象适配器）：Adapter 持有 Adaptee 实例并实现 Target 接口

角色：

- **Target**（目标）：定义客户类使用的与特定领域相关的接口
- **Adaptee**（被适配者）：定义一个被用来适配的已存在的接口
- **Adapter**（适配器）：适配 Adaptee 类中的接口至 Target 类中的接口
- **Client**（客户）：使用 Target 类中的接口

使用条件：

- 希望使用一个已存在的类，它的接口却与希望的接口不匹配
- 要创建一个可复用的类，该类可以与其它不相关的类或不可预见的类协同工作
- 想使用一些已存在的类，但不能对每一个类子类化以匹配它们的接口（仅适用于对象组合的适配器模式）

两种实现对比：

- **继承机制**：当想要匹配 Adaptee 类及其所有子类时，Adapter 类将不能胜任；Adapter 类可以重定义 Adaptee 类的行为；不需要间接指针以得到 adaptee 对象
- **对象组合**：允许一个 Adapter 类与多个 Adaptee 类一起工作；重定义 Adaptee 类的行为比较困难

9.3.2 组合模式(Composite Pattern)

[p.127-133]

结构：由三个角色组成：

- **Component**（部件）：为组合中的对象声明接口；在适当的情况下实现所有类共有接口的缺省行为；声明接口用于访问和管理组合对象中的子部件
- **Leaf**（叶子）：代表部件中的叶子对象，它没有子部件；定义组合对象中原始对象的行为
- **Composite**（组合）：定义有子部件的部件的行为；存储子部件；实现 Component 类接口中与子部件有关的操作

使用条件：

- 希望表示对象的整体/部分层次结构
- 希望一致地处理叶子对象和组合对象

使用效果：

- 定义了对对象的整体/部分层次结构，客户代码中任何用到叶子对象的地方均可使用组合对象
- 简化了客户代码，客户代码不必关心处理的是一个叶子对象还是组合对象
- 易于添加新的 Composite 和 Leaf 子类，客户代码不必因新的类而改变
- 使设计更加通用，但难于限制只组合某些特定的部件

9.4 行为型模式(Behavioral Patterns)

[p.134-149]

行为型模式不仅描述对象或类的模式，还描述它们之间的通信模式。这些模式刻画了在运行时难以跟踪的复杂的控制流。

行为型模式使设计者的注意力从控制流转移到对象间的联系方式上。

9.4.1 迭代器模式(Iterator Pattern)

[p.135-141]

结构：由以下角色组成：

- **Iterator**（迭代器接口）：定义访问和遍历聚合对象的接口（first(), next(), isDone(), currentItem()）
- **ConcreteIterator**（具体迭代器）：实现 Iterator 接口；在聚合对象遍历时跟踪当前位置
- **Aggregate**（聚合）：定义创建相应迭代器对象的接口
- **ConcreteAggregate**（具体聚合）：实现 Aggregate 接口，返回 ConcreteIterator 适当实例

使用条件：

- 在不暴露聚合对象内部表示的条件下访问聚集对象
- 支持对聚合对象的多种遍历
- 为遍历不同的聚合对象提供一个统一接口

使用效果：

- 支持以不同的方式遍历聚合对象
- 简化了聚合对象的接口
- 由于每个迭代器保持它自己的遍历状态，因此可以在同一个聚合对象进行多种遍历

9.4.2 观察者模式(Observer Pattern)

[p.142-149]

结构：由以下角色组成：

- **Subject**（主题）：知道其观察者；提供接口以连接观察者对象和解除连接（attach, detach, notify）
- **ConcreteSubject**（具体主题）：存储 ConcreteObserver 对象感兴趣的状态；当其状态发生变化时向它的所有观察者发送通知
- **Observer**（观察者接口）：为那些在主题发生改变时需要获得通知的对象定义一个 update 接口
- **ConcreteObserver**（具体观察者）：维护一个对 ConcreteSubject 对象的引用；存储一个与主题一致的状态；实现 Observer 的更新接口以使自身状态与主题的状态保持一致

使用条件：

- 一个模型有两个方面，其中一方面依赖于另一方面。将二者分别封装在独立的对象中可使它们可以各自独立地改变和复用
- 一个对象的改变需要同时改变其它对象，且不知道有多少对象需改变
- 一个对象必须能够在对别的对象一无所知的情况下通知它们

使用效果：

- 减少了 Subject 类和 Observer 类之间的耦合
- 支持广播通信。主题发送的通知不需指定它的接收者。处理还是忽略一个通知取决于观察者
- 可能会发生预想不到的更新

10 应用框架(Application Frameworks)

[p.150-158]

10.1 应用框架的概念

[p.150]

从复用的角度来看，类作为复用单元往往太小，而且类本身也很少完成有用的功能，需要与其它类一起协作。

应用框架(Application Framework)是一组协同类，用来为一族相关应用提供可复用的基础设施，包括设计和源代码。

示例：

- 微软的基础类库(Microsoft Foundation Class, MFC)：为 Windows 应用程序提供的创建 GUI 界面的框架
- SWING：Sun 公司提供的创建 Java GUI 应用界面的框架
- JUnit 框架：用于 Java 测试

10.2 应用框架的特点

[p.151]

1. **模块化(Modularity)**: 框架能将易变的实现细节隐藏在稳定的接口之中, 更加强化了模块化的特征
2. **复用性(Reusability)**: 应用框架的代码复用简化了应用的开发。同时, 应用框架也提供了可复用的抽象算法和高层设计以及交流所用的词汇 (分析)
3. **可扩展性(Extensibility)**: 应用框架常利用挂勾(Hook)方法来允许新的应用扩展框架的有关接口, 从而增强了框架的可扩展性
4. **控制倒置(Inversion of Control)**: 是应用框架而不是应用来决定应用的哪些具体方法响应有关外部事件

10.3 应用框架的分类

[p.152]

- **根据复用方式**: 白盒框架(White-box) (通过继承实现)、灰盒框架(Gray-box)、黑盒框架(Black-box) (通过组装实现)
- **根据涉及范围**: 系统基础设施框架、中间件集成框架、企业应用框架
- **根据关联程度**: 水平框架(Horizontal Framework) (横向框架)、垂直框架(Vertical Framework) (纵向框架)

10.4 应用框架的学习与使用

[p.153-154]

学习一个应用框架比学习常规类库要困难些:

- 首先, 一个框架是由一组协同工作的类构成, 因而在学习时不能一次只学习一个类, 而必须同时学习多个相关类
- 其次, 框架中一些重要的类大多是抽象类, 它们是不完整的, 需要实现具体的类, 从而学习者必须弄清哪些是框架中的不变部分, 哪些是需要完善的部分

学习方法: 从例子开始。选择一个框架明显适合的小应用, 考虑例子集中相似的例子, 然后拷贝它并实现一些简单的功能。单步跟踪应用, 学习有关的控制流。

10.5 框架的开发方法

[p.155]

- **自顶向下分析法**: 通过进行某种领域工程过程来开发框架, 首先进行领域分析, 然后实行领域的设计和实现
- **自底向上综合法**: 首先在框架领域内开发一个应用, 然后引入应用的变种来形成框架

10.6 MVC 框架(Model-View-Controller)

[p.156-158]

MVC(Model-View-Controller)框架结构是为那些需要为同一数据提供多个视图的应用程序而设计的, 它很好地实现了数据层与表示层的分离。

三个组成部分:

- **模型(Model)**: 管理系统的数据及相关操作

- **视图(View)**: 定义和管理数据如何呈现给用户
- **控制器(Controller)**: 管理用户交互（如键盘按键、鼠标点击等），并将这些交互传递给视图和模型

MVC 框架中类的交互流程:

1. 当用户进行一些输入动作后，控制器接收用户事件，并根据事件的类型来改变模型
2. 视图事先会在模型中登记，当模型数据发生改变时，马上通知已向此模型登记的每个视图
3. 视图从模型中取得最新的数据并刷新自己

MVC 框架适配:

- MVC 是一种通用的 GUI 框架
- Java SWING 是 MVC 框架的一种适配
- Smalltalk-80 v2.0 中也存在 MVC 框架的一种适配

10.7 库/设计模式/框架/体系结构复用比较

[p.159]

四种复用机制从控制程度和功能范围两个维度进行比较:

- **库(Library)**: 功能范围小，控制权在应用
- **设计模式(Design Pattern)**: 功能范围小，控制权在应用
- **框架(Framework)**: 功能范围较大，控制权在框架（控制倒置）
- **体系结构(Architecture)**: 功能范围最大，提供最高层次的复用

11 软件体系结构设计(Software Architecture Design)

[p.163-195]

体系结构设计用来定义系统的组成成分和它们之间的关系，为系统设计与实现建立一个框架。

Architectural design is a creative process so the process differs depending on the type of system being developed.（体系结构设计是一个创造性过程，因此过程因系统类型不同而异。）

11.1 体系结构与软件质量的关系

[p.164]

表 6: 体系结构设计对质量属性的影响

质量属性	体系结构设计对策
性能(Performance)	局部化关键操作并最小化通信。使用大型而非细粒度组件。
安全性(Security)	使用分层体系结构，将关键资产放在内层。
安全性(Safety)	将安全关键功能局部化在少量子系统中。
可用性(Availability)	包含冗余组件和容错机制。
可维护性(Maintainability)	使用细粒度、可替换的组件。

11.2 体系结构设计视图(Architecture Design Views)

[p.165-170]

体系结构设计视图：从某一视角或某一点上对系统的抽象。

11.2.1 4+1 视图模型

[p.165]

- **逻辑视图(Logical View)**: 整个系统的抽象结构表述，关注系统的功能需求
- **开发视图(Development View)**: 描述开发环境中软件系统组织结构，主要关注相关的内部需求，如文件、资源、编程语言约束等
- **进程视图(Process View)**: 捕捉设计的并发和同步特征，主要关注软件运行有关的非功能需求，如吞吐量、可用性、可扩展性等
- **物理视图(Physical View)**: 描述软件到硬件的映射，反映软件系统的分布式特性，主要关注部署有关的非功能需求，如物理拓扑结构、通信等
- **场景(Scenarios)**: 利用少量重要的场景将四个视图中的元素无缝集成在一起工作，关注最终用户需求

11.2.2 C4 模型视图

[p.166-170]

1. **系统上下文图(System Context Diagram)**: 反映系统与用户及其他软件系统之间的关系。图中颜色表示哪些软件系统已经存在（灰色）以及待构建的系统（蓝色）。[p.167]
2. **容器图(Container Diagram)**: 显示组成该软件系统的容器（应用程序、数据存储、微服务等）。如互联网银行系统由服务器端 Web 应用、客户端单页面应用、移动应用、服务器端 API 应用和数据库五个容器组成。[p.168]
3. **组件图(Component Diagram)**: 将单个容器放大，以显示其中的组件。如两个 Spring MVC REST 控制器为 JSON/HTTPS API 提供访问点。[p.169]
4. **代码图(Code Diagram)**: 表示组件由很多类组成，实现细节直接反映了代码。如 Mainframe-BankingSystemFacade 组件的代码元素（接口和类）。[p.170]

11.3 体系结构设计五原则

[p.171]

1. 稳定的软件体系结构，有助于软件系统的稳定性
2. 软件体系结构易理解，有助于开发人员以及系统其他涉众之间的交流，有利于控制系统的复杂性，也能够有效避免理解错误而引入故障
3. 在将系统划分成不同组成成分时，尽可能**复用已有的成分**，提高效率和质量
4. 综合考虑各种因素，**权衡利弊**做出决策
5. 把**不同的关注点分离**开来，分别进行处理，是处理复杂性的一个重要原则

11.4 体系结构设计六大任务

[p.172]

1. 体系结构需求一般有三个方面：系统质量需求、系统业务需求和系统开发约束
2. 利用优先级来决定哪些需求纳入体系结构设计（决策）
3. 为了降低体系结构设计风险，在设计时往往选择能满足高优先级体系结构需求且好理解的**成熟的体系结构模式**
4. 在确定体系结构组成成分时，除了考虑体系结构需求外，还应该注意各成分之间的**低耦合性**以及各组成成分的**易理解性和可复用性**
5. 将确定的体系结构组成成分分配到模式的某个部分，明确各组成成分之间的关系和约束，并形成**体系结构描述文档**
6. 评估的目的是确保提出的软件体系结构能够满足应用需求

11.5 体系结构模式风格(Architectural Pattern Styles)

[p.174-195]

软件体系结构设计模式是针对特定设计情况下反复出现的设计问题，提出的一种抽象的可重用设计方案，也称为**软件体系结构风格(Architectural Style)**。

软件体系结构设计模式的描述包括六个方面：模式的名称、模式的简要说明、使用模型的实例、使用模式的时机、模式的优点和缺点。

11.5.1 MVC 模式(Model-View-Controller)

[p.176-178]

表 7: MVC 体系结构模式

属性	描述
名称	MVC (Model-View-Controller)
描述	将表示和交互从系统数据中分离。系统被构造为三个相互交互的逻辑组件：Model 管理数据及相关操作；View 定义和管理数据的呈现；Controller 管理用户交互并传递给 View 和 Model。
使用时机	当有多种方法来查看和与数据交互时；当交互和数据显示的将来需求未知时。
优点	允许数据独立于其表示而变化，反之亦然。支持同一数据以不同方式呈现，一个表示的更改在所有视图中体现。
缺点	当数据模型和交互简单时，可能涉及额外的代码和代码复杂性。

Web 应用中的 MVC：控制器处理 HTTP 请求，模型管理业务逻辑和数据，视图负责呈现 HTML 页面。

11.5.2 层次模式(Layered Architecture)

[p.179-182]

表 8: 层次体系结构模式

属性	描述
描述	将系统组织为多个层次，每个层次具有相关功能。每一层为上层提供服务，最低层代表整个系统中可能使用的核心服务。
使用时机	在现有系统之上构建新设施时；开发分布在多个团队，每个团队负责一层功能时；需要多级安全时。
优点	只要维护接口，允许替换整个层。可在每一层提供冗余设施以增加系统可靠性。
缺点	实践中，提供层间干净分离往往困难，高层可能需要直接与更底层交互而非通过紧邻下层。多级服务请求解释可能导致性能问题。

银行系统的三层 C/S 模型：如果用户访问 Web 应用服务器量比较大，可以采用增加 Web 应用服务器和负载均衡等，并将 Web 服务独立，形成 Web 服务器和应用服务器，从而形成多层体系结构。

11.5.3 中心存储模式(Repository Architecture)

[p.183-185]

表 9: 中心存储体系结构模式

属性	描述
描述	系统中所有数据在中央存储库中管理，所有系统组件都可访问。组件不直接交互，仅通过存储库交互。
示例	编译器：每个软件工具生成信息，然后可供其他工具使用。
使用时机	有大量信息生成且需长时间存储的系统；数据驱动系统中，存储库中的数据触发动作或工具。
优点	组件独立——不需要知道其他组件的存在。一个组件所做的更改可以传播到所有组件。所有数据可一致管理（如备份同时进行）。
缺点	存储库是单点故障；通过存储库组织所有通信可能低效；跨多台计算机分发存储库可能困难。

编译器的中心存储模式：

- 词法分析器：读取程序源代码，识别词与符号
- 语法分析器：将单词根据语法组合成表达式等，判断其结构上有无错误
- 语义分析器：最重要的工作是类型检查，此外还有变量引用处理、类型名称处理等
- 优化器：对中间代码进行优化以产生高效的机器语言程序

11.5.4 客户/服务器模式(Client-Server Architecture)

[p.186-188]

表 10: C/S 体系结构模式

属性	描述
描述	系统功能组织为服务，每个服务由独立服务器提供。客户端是这些服务的使用者，访问服务器以使用它们。
使用时机	共享数据库中的数据需要从多个位置访问时；由于服务器可以复制，也可以用于系统负载可变时。
优点	服务器可跨网络分布。通用功能（如打印服务）可供所有客户端使用，不需要所有服务都实现。
缺点	每个服务都是单点故障，容易受到拒绝服务攻击或服务器故障的影响。性能可能不可预测，因为依赖于网络 and 系统。如果服务器由不同组织拥有，可能存在管理问题。

11.5.5 管道过滤器模式(Pipe and Filter Architecture)

[p.189-191]

表 11: 管道过滤器体系结构模式

属性	描述
描述	系统中的数据处理组织为每个处理组件（过滤器）是离散的，执行一种类型的数据转换。数据从一种组件流向另一个组件进行处理（如同管道）。
使用时机	常用于数据处理应用（批处理和基于事务的），其中输入在单独阶段被处理以生成相关输出。
优点	易于理解，支持转换复用。工作流风格匹配许多业务流程结构。通过添加转换进行演变是直接的。可实现为顺序或并发系统。
缺点	数据传输格式必须在通信转换之间达成一致。每个转换必须解析其输入并以约定格式输出。增加系统开销。

特点：功能转换处理其输入以产生输出。在 UNIX shell 中称为管道和过滤器模型。变体非常常见。当转换是顺序的，这是批处理顺序模型，广泛用于数据处理系统。不太适合交互式系统。

11.5.6 微内核模式(Microkernel Architecture)

[p.192-193]

表 12: 微内核体系结构模式

属性	描述
描述	也称为插件模式(Plug-in Pattern)，由内核系统和插件组成。内核系统提供系统运行所需的最小功能集（如插件管理、插件连接和插件通信）。插件是独立的组件，包含特定的业务处理，用来增强或扩展内核系统。
示例	Eclipse 平台微内核模式体系结构。Eclipse 平台运行时为整个系统内核，所有业务功能（工作区、工作台、团队、调试和帮助）都用插件实现。
使用时机	系统要求在不同情况下需要不同功能时。将所有情况都需要的功能做成内核系统，不同情况需要的功能做成插件。
优点	简单性。内核由运行系统的最小功能集构成，插件的功能是隔离的，可独立加载和卸载。可增量式开发插件，逐步增加系统功能。可扩展性好。
缺点	内核系统难以做成分布式系统，导致伸缩性较差。内核出问题则整个系统不能使用。每个功能做成插件会增加开发复杂性（需处理插件与内核的通信、插件的注册等）。

11.5.7 微服务模式(Microservices Architecture)

[p.194-195]

表 13: 微服务体系结构模式

属性	描述
描述	将软件系统分解为可独立部署的自治单元，通过轻量级、与语言无关的方式进行通信，共同实现业务目标。主要由 API 层和微服务构成。API 层是客户端的入口点，客户端必须通过 API 层调用微服务。
示例	Spring Cloud Netflix: 服务网关 Zuul 提供路由、过滤、权限验证；Eureka 为注册中心；Config 为配置中心。原子服务层进行具体业务逻辑处理；聚合层聚合服务，绘制界面和接收用户请求。也可使用 SpringCloud Alibaba (Nacos)。
使用时机	需要高可用、高性能、高扩展的微服务系统。
优点	去中心化。每个微服务单独部署、独立运行。开发技术可各不相同。方便独立开发、容易实现持续交付、具有很好的弹性和可扩展性。重构、替换等不会影响其他微服务，便于维护，可靠性好。
缺点	运维复杂性增加，服务越多，潜在风险越多，失败可能越多。领域边界不清晰导致服务间依赖关系，使独立部署复杂化。跨服务功能开发复杂性大大增加（服务可能部署到不同机器甚至远程机器）。

12 软件体系结构示例：铁路客运专线信息系统

[p.197-206]

12.1 案例概述

[p.197-201]

铁路客运专线信息系统是一个大型综合信息系统，其体系结构设计包括：

- 概念模型：定义系统的核心概念及其关系
- 总体架构：包括业务系统、电子商务平台等多个子系统
- 电子商务平台架构：为旅客提供票务服务、信息查询等功能

12.2 网络平台设计

[p.202-206]

- 调度中心网络平台：负责列车运行的统一调度和指挥
- 调度所网络平台：各地方调度所的网络基础设施
- 资源共享网络平台：实现系统间的资源共享
- 信息安全保障网络平台：保障整个系统的信息安全

13 本章核心概念总结

13.1 关键术语对照表

表 14: 第 6 章核心术语中英对照表

中文术语	英文术语	中文术语	英文术语
概要设计	Preliminary Design	模块独立性	Module Independence
详细设计	Detailed Design	内聚度	Cohesion
抽象	Abstraction	耦合度	Coupling
求精	Refinement	功能内聚	Functional Cohesion
模块化	Modularity	数据耦合	Data Coupling
信息隐藏	Information Hiding	内容耦合	Content Coupling
软件结构图	Software Structure Chart (SC)	变换分析	Transform Analysis
层次图	Hierarchy Chart (H 图)	事务分析	Transaction Analysis
HIPO 图	Hierarchy plus IPO	扇出/扇入	Fan-out / Fan-in
IDEF0	Integration Definition 0	控制域	Scope of Control
单一职责原则	Single Responsibility Principle	作用域	Scope of Effect
开闭原则	Open-Closed Principle	设计模式	Design Pattern
里氏替换原则	Liskov Substitution Principle	工厂方法	Factory Method
依赖倒置原则	Dependency Inversion Principle	抽象工厂	Abstract Factory
接口隔离原则	Interface Segregation Principle	适配器模式	Adapter Pattern
组装复用原则	Composition/Aggregation Reuse Principle	组合模式	Composite Pattern
面向对象设计	Object-Oriented Design	观察者模式	Observer Pattern
问题域部分	Problem Domain Component	迭代器模式	Iterator Pattern
人机交互部分	Human-Computer Interaction Component	MVC 框架	Model-View-Controller
任务管理部分	Task Management Component	软件体系结构	Software Architecture
数据管理部分	Data Management Component	微服务模式	Microservices Architecture

13.2 学习要点提示

- 软件设计与质量模型：**理解 McCall、Boehm、ISO/IEC 9126 三种质量模型的演进，掌握软件产品质量公式
- 设计基本原理：**抽象与求精是软件设计的两大思维工具；模块化、信息隐藏是结构化设计的基础
- 内聚与耦合：**这是本章最核心的评判标准——高内聚低耦合是优秀软件设计的终极目标。需要牢记七种内聚（功能 > 信息 > 通信 > 过程 > 时间 > 逻辑 > 偶然）和七种耦合（非直接 < 数据 < 标记 < 控制 < 外部 < 公共 < 内容）的等级顺序
- 结构化设计方法：**区分变换型与事务型数据流图，掌握变换分析法和事务分析法的第一级、第

二级分解过程

5. **SC 图与 HIPO 图**：理解软件结构图中的模块类型、调用关系和信息传递表示
6. **类设计六大原则**：SRP（一个类一个职责）、OCP（对扩展开放对修改关闭）、LSP（子类型可替换基类型）、DIP（依赖抽象不依赖具体）、ISP（接口专一化）、CARP（组合优于继承）
7. **OOD 四大组成**：问题域、人机交互、任务管理、数据管理各有专门的设计考虑
8. **GoF 23 种模式**：创建型 5 种、结构型 7 种、行为型 11 种，每种模式理解其动机、结构和适用场景
9. **框架与 MVC**：理解框架的控制倒置特性，MVC 的数据层与表示层分离思想
10. **体系结构模式**：MVC、层次、中心存储、C/S、管道过滤器、微内核、微服务——每种模式都有其适用场景和权衡